Programming Assignment: Tomasulo Algorithm with Multiple Issue

Umar Ahmad

1. Introduction

The starting point was the provided Python implementation of Tomasulo's algorithm. It issues one instruction per cycle and has a ROB, register renaming, a load/store queue and a CDB. This assignment adds support for issuing **1 to 4 instructions per cycle, in order**. The issue width is set in the input file (`Issue width = n`, default 1) or on the command line (`-w n`).

To do that, and to make the simulator match the processor described in the handout, I made these changes:

1. **Multiple issue.** Up to W instructions are issued per cycle, strictly in program order. Issue stops at the first instruction that can't be issued.
2. **Multiple commit.** Up to W instructions commit per cycle, in order, from the head of the ROB.
3. **Input file parser.** The hardware configuration, ROB size, issue width, initial register and memory values, and the program are all read from the input file. Instructions are case-insensitive and commas are optional. In the starter code all of this was hard-coded in `main.py`.
4. **Branch prediction and speculation.** I added a 1-bit predictor with an 8-entry BTB indexed by the 3 LSBs of the word address, speculative fetch past branches, and misprediction recovery (recover the RAT, clear the RS/LSQ entries of wrong-path instructions, clear the younger ROB entries; the correct path is fetched at cycle n+2). The starter code had no prediction: it stopped fetching at every branch. I also added `Beq`.
5. **Load/store queue.** Store-to-load forwarding is checked in MEM and takes 1 cycle. Stores write memory only at commit. Memory is single-ported and non-pipelined.
6. **Output.** The instruction status table, register values and non-zero memory, in the format the handout asks for.
7. **Testing.** Eight test programs, a reference functional simulator, and a test script that checks the final state and a set of timing rules for every issue width.

What works

- Issue widths 1, 2, 3 and 4, with in-order issue and in-order commit.
- Every test produces the same final registers and memory, and the same committed-instruction sequence, as the in-order reference simulator. This holds at all four issue widths and with 1 to 4 CDBs (64 checked runs).
- Register renaming (RAW/WAR/WAW), the ROB, speculative execution with branch prediction and recovery, store-to-load forwarding, and memory disambiguation.
- Parameterizable configuration: number of RS / LSQ entries, EX cycles, memory cycles, number of FUs per type, ROB size, issue width, number of CDBs.

Limitations / simplifications

- **Conservative memory disambiguation.** A load waits until the addresses of all older stores are known. It never speculatively bypasses an unknown store address, so loads never need to be replayed.
- **Squashed ops still occupy their unit.** A wrong-path operation that is already inside a functional unit keeps the unit busy until it finishes; only its result is dropped.
- **Stores free the ROB at commit.** A store writes memory at commit and occupies the memory port for "Cycles in Mem" cycles. Its ROB entry is freed in the commit cycle, so younger instructions can keep committing while the write finishes.
- **No BTB tags.** As in the handout, the BTB is indexed only by the 3 LSBs, so two branches can alias (test 5 shows this).

2. Code changes

The simulator keeps the structure of the starter code: one file per pipeline stage (`issue.py`, `exe.py`, `mem.py`, `wb.py`, `commit.py`), called from a loop in `main.py`. I rewrote a large part of every stage, because the starter code had to change for multiple issue and also had several bugs:

- Its entries were `namedtuple` classes with attributes set on the class object.
- `check_if_sd_committable` was referenced but never called, so it was always true.
- It used the field name `ldsd_tag` instead of `ld_sd_tag`.
- It had no `Beq`.
- Its output had no register or memory values.

All processor state now lives in one `CPU` object (`init.py`). Each value carries the cycle from which it may be used. Because of that, the order in which the stage functions are called inside a cycle doesn't change the timing.

2.1 Main loop (`main.py`)

```
        while True:
            cpu.cycle += 1
            commit(cpu)
            wb(cpu)
            mem(cpu)
            exe(cpu)
            issue(cpu)
            recover(cpu)          # branch misprediction recovery (end of cycle)
            fetch_done = not 0 <= cpu.pc < len(program)
            if fetch_done and not cpu.rob and cpu.cycle >= cpu.fetch_resume:
                break
```

2.2 Issue (`issue.py`)

The starter `issue()` fetched one instruction. The new version loops up to `issue_width` times. It stops at the first instruction that can't be issued (ROB, RS or LSQ full), so instructions always issue in order. It also stops after a predicted-taken branch, because the next instruction comes from the branch target in the next cycle.

```
        def issue(cpu):
            if cpu.cycle < cpu.fetch_resume:
                return                                  # recovering from a misprediction
            for _ in range(cpu.cfg.issue_width):
                if not 0 <= cpu.pc < len(cpu.program):
                    break
                ins = cpu.program[cpu.pc]
                entry = issue_one(cpu, ins)             # ROB + RS/LSQ + RAT
                if entry is None:
                    break                               # structural stall: stay in order
                if ins.op in BRANCHES:
                    taken, nxt = predict(cpu, cpu.pc, ins)   # 1-bit predictor + BTB
                    entry.pred_taken, entry.pred_next = taken, nxt
                    cpu.pc = nxt
                    if taken:
                        break                           # fetch group ends at a taken branch
                else:
                    cpu.pc += 1
```

`issue_one` reads the sources through the RAT *before* renaming the destination, so an instruction such as `Addi R1, R1, 4` works. Each source operand gets one of three things:

- the register file value (RAT empty),
- the value already broadcast into the ROB, or
- the ROB tag to wait for.

Several instructions issued in the same cycle are processed in order, so a younger instruction sees the RAT update made by an older one in the same group. This is what makes dependent instructions issued together work.

2.3 Execute (`exe.py`, `branch.py`)

Each cycle, every unit type starts the oldest ready instruction(s) on its free units:

- The integer adder is unpipelined.

- The FP adder and FP multiplier are pipelined.
- Loads and stores use the dedicated address adder of the load/store unit.

An instruction can start EX in the cycle after it was issued and in the cycle after its operands were broadcast. Its RS entry is freed when it starts.

Branches run on the integer adder and are resolved at the end of EX (`resolve_branch`). Resolution updates the BTB entry (1-bit outcome and target). If the actual next PC differs from the predicted one, the branch is marked mispredicted. At the end of the cycle, `recover()` squashes everything younger than the oldest mispredicted branch:

```
        # 3. clear ROB entries younger than the branch
        for e in cpu.rob:
            if e.seq > br.seq:
                e.squashed = True
        # 2. clear RS entries (and their wait-for tags), LSQ entries, pending results
        # 1. recover the RAT from the surviving ROB entries
        cpu.pc = br.actual_next
        cpu.fetch_resume = cpu.cycle + 2     # mispredict detected in n -> fetch in n+2
```

2.4 Memory (`mem.py`)

Loads are handled oldest first, starting the cycle after their address is known:

- A load waits until the addresses of all older stores in the queue are known.
- If the nearest older store has the same address, its data is forwarded in 1 cycle. If that data isn't ready yet, the load waits.
- Otherwise the load accesses memory, if the single memory port is free.

A load's queue entry is cleared once it has its value. A store stays in the queue until it commits. During its commit cycle it is still visible for forwarding.

2.5 Write-back (`wb.py`)

The starter version moved one result from the results buffer to the CDB per cycle and broadcast it in the *next* cycle. That added a cycle. The new version broadcasts the oldest ready results directly, one per CDB per cycle. The handout specifies one CDB, which is the default. The number can be changed with `CDB = n` for the experiment in section 4.3.

```
        def wb(cpu):
            ready = sorted((r for r in cpu.results_buffer if r.ready <= cpu.cycle),
                           key=lambda r: r.entry.seq)
            for r in ready[:cpu.cfg.num_cdb]:
                broadcast(cpu, r.entry, r.value)     # ROB value + all waiting RS / LSQ operands
                cpu.results_buffer.remove(r)
```

2.6 Commit (`commit.py`)

The starter `commit()` committed one instruction per cycle (two in a special case after a store). The new version commits up to `issue_width` instructions per cycle from the ROB head and stops at the first one that isn't ready:

- An ALU op or load is ready once it was broadcast in an earlier cycle.
- A branch is ready once it was resolved in an earlier cycle.
- A store is ready once its address and data are known and the memory port is free.

A committing store writes memory and occupies the port for "Cycles in Mem" cycles, so at most one store commits per cycle. Committing writes the architectural register and clears the RAT entry if it still points to this instruction.

```
        for _ in range(cpu.cfg.issue_width):
            if not cpu.rob or not ready_to_commit(cpu, cpu.rob[0]):
                break
            entry = cpu.rob.popleft()
            ...  # store -> memory, otherwise -> architectural register; clear RAT
            entry.commit = cpu.cycle
```

2.7 Input parser and output (`init.py`, `print_status.py`)

`parse_input()` reads the handout's input format: the unit table (the header line is skipped), `ROB entries = n`, `Issue width = n`, the initial values `R1=10, F2=30.1` and `Mem[4]=1`, and the program. The output is the instruction status table, all 64 register values, the non-zero memory words, and a summary (cycles, IPC, mispredictions, squashed instructions).

3. Timing model

Stage	Rule
ISSUE	Up to W per cycle, in order. Needs a free ROB entry and RS (or LSQ) entry. Stops after a predicted-taken branch. Nothing is issued in cycle n+1 after a misprediction detected in cycle n.
EX	Starts at the earliest in issue+1, and in the cycle after the operands' WB. Lasts "Cycles in EX". Integer adder unpipelined; FP adder and FP multiplier pipelined; loads/stores use the address adder.
MEM (loads)	Starts at the earliest in the cycle after EX ends. Forwarding takes 1 cycle; a memory access takes "Cycles in Mem" on the single, non-pipelined port.

Stage	Rule
WB	The cycle after EX ends (ALU) or after MEM ends (loads), if a CDB is free. The oldest instruction wins.
COMMIT	The cycle after WB (or after branch resolution), up to W per cycle, in order. A store writes memory at commit (shown in its MEM column).
Branch	Predicted at issue, resolved at the end of EX (cycle n). Recovery in n+1, correct fetch in n+2.

A resource freed in cycle n (a ROB entry by COMMIT, an RS entry by the start of EX, an LSQ entry when a load gets its value) can be used by an instruction issued in cycle n.

4. Test cases and results

4.1 How correctness is checked

`python run_tests.py` runs every test in `tests/` with issue widths 1, 2, 3 and 4. Each run is checked in two ways.

Functional check. `code/reference.py` is a simple in-order interpreter. The simulator's final registers and memory, and its sequence of committed instructions, must equal the reference.

Timing checks. Each run must also satisfy these rules:

- In-order issue with at most W instructions per cycle.
- In-order commit with at most W per cycle.
- At most one broadcast per CDB per cycle.
- EX starts after ISSUE and after every producer's WB.
- For loads, MEM comes after EX.
- WB comes after EX/MEM, and COMMIT after WB.
- The memory port is never used by two accesses at once.
- The integer adder is never over-subscribed.
- After a misprediction resolved in cycle n, no correct-path instruction issues before n+2.
- The ROB limit is respected.

All 32 runs (8 tests × 4 widths) pass both checks. The 32 extra runs with more CDBs (section 4.3) pass as well. The full output of every run is in `results/`.

4.2 Summary of results (1 CDB, as specified)

Total cycles / IPC for each issue width:

Test	W=1 cycles / IPC	W=2	W=3	W=4
test0_starter_program	57 / 0.44	54 / 0.46	54 / 0.46	54 / 0.46
test1_independent	18 / 0.67	15 / 0.80	15 / 0.80	15 / 0.80
test2_dependencies	30 / 0.40	30 / 0.40	30 / 0.40	30 / 0.40
test3_memory	32 / 0.41	31 / 0.42	31 / 0.42	31 / 0.42
test4_loop	68 / 0.74	59 / 0.85	56 / 0.89	55 / 0.91
test5_branches	31 / 0.61	30 / 0.63	30 / 0.63	30 / 0.63
test6_structural	31 / 0.35	30 / 0.37	30 / 0.37	30 / 0.37
test7_handout_sample	39 / 0.23	35 / 0.26	33 / 0.27	33 / 0.27

Observations:

- **Multiple issue helps when there is independent work at the front end.** The loop in test 4 goes from 68 cycles at W=1 to 55 cycles at W=4 (IPC 0.74 to 0.91). Test 1 goes from 18 cycles to 15.
- **With one CDB, IPC can't go above about 1.** Only one result can be written back per cycle. All 12 instructions of test 1 produce a result, so write-back takes 12 cycles (3 to 14) whatever the issue width. This is why widths 2, 3 and 4 give the same time there. Section 4.3 shows what happens with more CDBs.
- **Dependence chains don't speed up.** Test 2 is limited by its RAW chain, so wider issue only lets instructions wait longer in the reservation stations; it takes 30 cycles at every width.
- **Structural limits cap the gain.** Test 6 has a 4-entry ROB and one RS per unit, so it stalls on structural hazards whatever the width.
- **Branches limit the gain.** In tests 0, 5 and 7 the 2-cycle misprediction penalty and the end of the fetch group at every predicted-taken branch limit what a wider front end can do.
- The final architectural state is the same for every width, as it must be.

4.3 Extra experiment: number of CDBs = issue width

With W CDBs, test 1 scales from IPC 0.67 to 1.33 and test 4 from 0.74 to 1.16. This confirms that the multi-issue front end and the multi-commit back end work, and that the single CDB is the bottleneck in section 4.2.

Test	W=1, 1 CDB	W=2, 2 CDBs	W=3, 3 CDBs	W=4, 4 CDBs
test0_starter_program	57 / 0.44	54 / 0.46	54 / 0.46	54 / 0.46
test1_independent	18 / 0.67	12 / 1.00	10 / 1.20	9 / 1.33

Test	W=1, 1 CDB	W=2, 2 CDBs	W=3, 3 CDBs	W=4, 4 CDBs
test2_dependencies	30 / 0.40	30 / 0.40	30 / 0.40	30 / 0.40
test3_memory	32 / 0.41	31 / 0.42	31 / 0.42	31 / 0.42
test4_loop	68 / 0.74	48 / 1.04	44 / 1.14	43 / 1.16
test5_branches	31 / 0.61	30 / 0.63	30 / 0.63	30 / 0.63
test6_structural	31 / 0.35	30 / 0.37	30 / 0.37	30 / 0.37
test7_handout_sample	39 / 0.23	35 / 0.26	33 / 0.27	33 / 0.27

4.4 Test cases in detail

For every test, the program and configuration are listed with the final state. The final state is identical for every issue width and matches the reference simulator. Full instruction status tables are shown for several widths. Tables for all widths are in `results/<test>_w<W>.txt`.

Test 0: Starter program (`tests/test0_starter_program.txt`)

The program that came with the starter code (`code.in`), converted to the new input format. It is a 3-iteration loop with loads, a long-latency multiply, a store and a backward branch.

```
                # of rs     Cycles in EX    Cycles in Mem   # of FUs
Integer adder     4             1                             1
FP adder          3             4                             1
FP multiplier     2             15                            1
Load/store unit   5             1               5             1

ROB entries = 64
Issue width = 2
R1=12, R2=32, F20=3.0
Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=6.0, Mem[28]=5.0, Mem[32]=4.0

Ld F2, 0(R1)
Mult.d F4, F2, F20
Ld F6, 0(R2)
Add.d F6, F4, F6
Sd F6, 0(R2)
Addi R1, R1, -4
Addi R2, R2, -4
Bne R1, R0, -8
Add.d F20, F2, F2
```

Final state (all widths): R2=20, F2=3.0, F4=9.0, F6=15.0, F20=6.0.
Non-zero memory: Mem[4]=3.0, Mem[8]=2.0, Mem[12]=1.0, Mem[24]=15.0, Mem[28]=11.0, Mem[32]=7.0.

Issue width 1

```
                          ISSUE   EX       MEM      WB      COMMIT
Ld F2, 0(R1)              1       2        3-7      8       9
Mult.d F4, F2, F20        2       9-23              24      25
Ld F6, 0(R2)              3       4        8-12     13      26
Add.d F6, F4, F6          4       25-28             29      30
Sd F6, 0(R2)              5       6        34-38            34
Addi R1, R1, -4           6       7                 9       35
Addi R2, R2, -4           7       8                 10      36
Bne R1, R0, -8            8       10                        37
Ld F2, 0(R1)              12      13       14-18    19      38
Mult.d F4, F2, F20        13      20-34             35      39
Ld F6, 0(R2)              14      15       19-23    25      40
Add.d F6, F4, F6          15      36-39             40      41
Sd F6, 0(R2)              16      17       42-46            42
Addi R1, R1, -4           17      18                20      43
Addi R2, R2, -4           18      19                21      44
Bne R1, R0, -8            19      21                        45
Ld F2, 0(R1)              20      21       24-28    30      46
Mult.d F4, F2, F20        21      31-45             46      47
Ld F6, 0(R2)              22      23       29-33    34      48
Add.d F6, F4, F6          23      47-50             51      52
Sd F6, 0(R2)              24      25       53-57            53
Addi R1, R1, -4           25      26                27      54
Addi R2, R2, -4           26      27                28      55
Bne R1, R0, -8            27      28                        56
Add.d F20, F2, F2         30      31-34             36      57

Total cycles: 57   IPC: 0.44   mispredictions: 2
```

Issue width 2

```
                          ISSUE   EX       MEM      WB      COMMIT
Ld F2, 0(R1)              1       2        3-7      8       9
Mult.d F4, F2, F20        1       9-23              24      25
Ld F6, 0(R2)              2       3        8-12     13      25
Add.d F6, F4, F6          2       25-28             29      30
Sd F6, 0(R2)              3       4        33-37            33
Addi R1, R1, -4           3       4                 5       33
Addi R2, R2, -4           4       5                 6       34
Bne R1, R0, -8            4       6                         34
Ld F2, 0(R1)              8       9        13-17    18      35
Mult.d F4, F2, F20        8       19-33             34      35
Ld F6, 0(R2)              9       10       18-22    23      36
Add.d F6, F4, F6          9       35-38             39      40
Sd F6, 0(R2)              10      11       40-44            40
Addi R1, R1, -4           10      11                12      41
Addi R2, R2, -4           11      12                14      41
Bne R1, R0, -8            11      13                        42
Ld F2, 0(R1)              13      14       23-27    28      42
Mult.d F4, F2, F20        13      29-43             44      45
Ld F6, 0(R2)              18      19       28-32    33      45
Add.d F6, F4, F6          18      45-48             49      50
Sd F6, 0(R2)              23      24       50-54            50
Addi R1, R1, -4           23      24                25      51
Addi R2, R2, -4           24      25                26      51
Bne R1, R0, -8            24      26                        52
Add.d F20, F2, F2         28      29-32             35      52

Total cycles: 54   IPC: 0.46   mispredictions: 2
```

Test 1: Independent instructions (`tests/test1_independent.txt`)

12 instructions with no data dependences, spread over all functional units. This test shows how much the wider issue stage helps when there is plenty of ILP.

```
               # of rs     Cycles in EX    Cycles in Mem   # of FUs
Integer adder     4             1                           2
FP adder          4             2                           1
FP multiplier     4             4                           1
Load/store unit   4             1               2           1

ROB entries = 32
Issue width = 1   (overridden: 1, 2, 3, 4)
R1=10, R2=20, F1=1.5, F2=2.5
Mem[0]=7, Mem[4]=8

Add R3, R1, R2
Addi R4, R1, 5
Add.d F3, F1, F2
Mult.d F4, F1, F2
Sub R5, R2, R1
Ld F9, 0(R0)
Sub.d F5, F2, F1
Mult.d F6, F2, F2
Addi R6, R2, -3
Add R7, R1, R1
Add.d F7, F1, F1
Mult.d F8, F1, F1
```

Final state (all widths): R1=10, R2=20, R3=30, R4=15, R5=10, R6=17, R7=20, F1=1.5, F2=2.5, F3=4.0, F4=3.75, F5=1.0, F6=6.25, F7=3.0, F8=2.25, F9=7.0.
Non-zero memory: Mem[0]=7.0, Mem[4]=8.0.

Issue width 1

```
                          ISSUE   EX       MEM      WB      COMMIT
Add R3, R1, R2            1       2                3       4
Addi R4, R1, 5            2       3                4       5
Add.d F3, F1, F2          3       4-5              6       7
Mult.d F4, F1, F2         4       5-8              9       10
Sub R5, R2, R1            5       6                7       11
Ld F9, 0(R0)              6       7        8-9     10      12
Sub.d F5, F2, F1          7       8-9              11      13
Mult.d F6, F2, F2         8       9-12             13      14
Addi R6, R2, -3           9       10               12      15
Add R7, R1, R1            10      11               14      16
Add.d F7, F1, F1          11      12-13            15      17
Mult.d F8, F1, F1         12      13-16            17      18

Total cycles: 18   IPC: 0.67   mispredictions: 0
```

Issue width 2

```
                          ISSUE   EX       MEM      WB      COMMIT
Add R3, R1, R2            1       2                3       4
Addi R4, R1, 5            1       2                4       5
Add.d F3, F1, F2          2       3-4              5       6
Mult.d F4, F1, F2         2       3-6              7       8
Sub R5, R2, R1            3       4                6       8
Ld F9, 0(R0)              3       4        5-6     8       9
Sub.d F5, F2, F1          4       5-6              9       10
Mult.d F6, F2, F2         4       5-8              10      11
Addi R6, R2, -3           5       6                11      12
Add R7, R1, R1            5       6                12      13
Add.d F7, F1, F1          6       7-8              13      14
Mult.d F8, F1, F1         6       7-10             14      15

Total cycles: 15   IPC: 0.80   mispredictions: 0
```

Issue width 4

```
                          ISSUE   EX       MEM      WB      COMMIT
Add R3, R1, R2            1       2                 3       4
Addi R4, R1, 5            1       2                 4       5
Add.d F3, F1, F2          1       2-3               5       6
Mult.d F4, F1, F2         1       2-5               6       7
Sub R5, R2, R1            2       3                 7       8
Ld F9, 0(R0)              2       3        4-5      8       9
Sub.d F5, F2, F1          2       3-4               9       10
Mult.d F6, F2, F2         2       3-6               10      11
Addi R6, R2, -3           3       4                 11      12
Add R7, R1, R1            3       4                 12      13
Add.d F7, F1, F1          3       4-5               13      14
Mult.d F8, F1, F1         3       4-7               14      15

Total cycles: 15   IPC: 0.80   mispredictions: 0
```

Test 2: RAW, WAR and WAW dependences (`tests/test2_dependencies.txt`)

RAW chains, plus a second write of F2 (WAW) that happens while Mult.d still has to read the old F2 (WAR). Renaming must give Mult.d the old value and Sub.d the new one.

```
                # of rs     Cycles in EX    Cycles in Mem   # of FUs
Integer adder     3             1                             1
FP adder          3             3                             1
FP multiplier     2             5                             1
Load/store unit   3             1               3             1

ROB entries = 16
Issue width = 1   (overridden: 1, 2, 3, 4)
F0=1.0, F1=2.0, R1=5

Add.d F2, F0, F1
Mult.d F3, F2, F2
Add.d F2, F1, F1
Sub.d F4, F3, F2
Addi R1, R1, 1
Addi R1, R1, 1
Add R2, R1, R1
Sub R3, R2, R1
Mult.d F5, F4, F2
Add.d F6, F5, F3
Sub R1, R3, R1
Add.d F2, F6, F2
```

Final state (all widths): R2=14, R3=7, F0=1.0, F1=2.0, F2=33.0, F3=9.0, F4=5.0, F5=20.0, F6=29.0.
Non-zero memory: none.

Issue width 1

```
                          ISSUE   EX       MEM      WB      COMMIT
Add.d F2, F0, F1          1       2-4              5       6
Mult.d F3, F2, F2         2       6-10             11      12
Add.d F2, F1, F1          3       4-6              7       13
Sub.d F4, F3, F2          4       12-14            15      16
Addi R1, R1, 1            5       6                8       17
Addi R1, R1, 1            6       9                10      18
Add R2, R1, R1            7       11               12      19
Sub R3, R2, R1            8       13               14      20
Mult.d F5, F4, F2         9       16-20            21      22
Add.d F6, F5, F3          10      22-24            25      26
Sub R1, R3, R1            11      15               16      27
Add.d F2, F6, F2          12      26-28            29      30

Total cycles: 30   IPC: 0.40   mispredictions: 0
```

Issue width 4

```
                          ISSUE   EX       MEM      WB      COMMIT
Add.d F2, F0, F1          1       2-4              5       6
Mult.d F3, F2, F2         1       6-10             11      12
Add.d F2, F1, F1          1       3-5              6       12
Sub.d F4, F3, F2          1       12-14            15      16
Addi R1, R1, 1            2       3                4       16
Addi R1, R1, 1            2       5                7       16
Add R2, R1, R1            2       8                9       16
Sub R3, R2, R1            3       10               12      17
Mult.d F5, F4, F2         3       16-20            21      22
Add.d F6, F5, F3          3       22-24            25      26
Sub R1, R3, R1            5       13               14      26
Add.d F2, F6, F2          5       26-28            29      30

Total cycles: 30   IPC: 0.40   mispredictions: 0
```

Test 3: Load/store queue (`tests/test3_memory.txt`)

Store-to-load forwarding, loads that bypass stores to other addresses, a load that waits for a store’s data (produced by a 6-cycle multiply), and stores that write memory only at commit.

```
               # of rs     Cycles in EX    Cycles in Mem   # of FUs
Integer adder     2             1                            1
FP adder          3             3                            1
FP multiplier     2             6                            1
Load/store unit   6             1               4            1

ROB entries = 32
Issue width = 1   (overridden: 1, 2, 3, 4)
R1=0, R2=8, F1=1.5, F2=2.0
Mem[0]=10, Mem[4]=20, Mem[8]=30, Mem[12]=40

Sd F1, 0(R1)
Ld F3, 0(R1)
Ld F4, 4(R1)
Add.d F5, F3, F4
Sd F5, 4(R2)
Ld F6, 12(R1)
Mult.d F7, F1, F2
Sd F7, 0(R2)
Ld F8, 8(R1)
Addi R3, R1, 16
Sd F8, 0(R3)
Ld F9, 16(R0)
Add.d F10, F9, F6
```

Final state (all widths): R2=8, R3=16, F1=1.5, F2=2.0, F3=1.5, F4=20.0, F5=21.5, F6=21.5, F7=3.0, F8=3.0, F9=3.0, F10=24.5.
Non-zero memory: Mem[0]=1.5, Mem[4]=20.0, Mem[8]=3.0, Mem[12]=21.5, Mem[16]=3.0.

Issue width 1

```
                          ISSUE   EX       MEM      WB      COMMIT
Sd F1, 0(R1)              1       2        3-6              3
Ld F3, 0(R1)              2       3        7-10     11      12
Ld F4, 4(R1)              3       4        11-14    15      16
Add.d F5, F3, F4          4       16-18             19      20
Sd F5, 4(R2)              5       6        21-24            21
Ld F6, 12(R1)             6       7        20       21      22
Mult.d F7, F1, F2         7       8-13              14      23
Sd F7, 0(R2)              8       9        25-28            25
Ld F8, 8(R1)              9       10       15       16      26
Addi R3, R1, 16           10      11                12      27
Sd F8, 0(R3)              11      13       29-32            29
Ld F9, 16(R0)             15      16       17       18      30
Add.d F10, F9, F6         16      22-24             25      31

Total cycles: 32   IPC: 0.41   mispredictions: 0
```

Issue width 2

```
                          ISSUE   EX       MEM      WB      COMMIT
Sd F1, 0(R1)              1       2        3-6              3
Ld F3, 0(R1)              1       3        7-10     11      12
Ld F4, 4(R1)              2       4        11-14    15      16
Add.d F5, F3, F4          2       16-18             19      20
Sd F5, 4(R2)              3       5        20-23            20
Ld F6, 12(R1)             3       6        20       21      22
Mult.d F7, F1, F2         4       5-10              12      22
Sd F7, 0(R2)              4       7        24-27            24
Ld F8, 8(R1)              5       8        13       14      24
Addi R3, R1, 16           5       6                 7       25
Sd F8, 0(R3)              11      12       28-31            28
Ld F9, 16(R0)             14      15       16       17      28
Add.d F10, F9, F6         14      22-24             25      29

Total cycles: 31   IPC: 0.42   mispredictions: 0
```

Test 4: Loop with branch prediction (`tests/test4_loop.txt`)

An 8-iteration loop that computes the sum and the sum of squares of an array. The 1-bit predictor mispredicts the first iteration (empty BTB, so it predicts not-taken) and the loop exit. Every other iteration is fetched speculatively.

```
                # of rs     Cycles in EX    Cycles in Mem   # of FUs
Integer adder     3             1                             1
FP adder          4             2                             1
FP multiplier     3             4                             1
Load/store unit   4             1               2             1

ROB entries = 32
Issue width = 1   (overridden: 1, 2, 3, 4)
R1=0, R2=32
Mem[0]=1, Mem[4]=2, Mem[8]=3, Mem[12]=4, Mem[16]=5, Mem[20]=6, Mem[24]=7, Mem[28]=8

Ld F1, 0(R1)
Add.d F0, F0, F1
Mult.d F2, F1, F1
Add.d F3, F3, F2
Addi R1, R1, 4
Bne R1, R2, -6
Sd F0, 0(R2)
Sd F3, 4(R2)
```

Final state (all widths): R1=32, R2=32, F0=36.0, F1=8.0, F2=64.0, F3=204.0.
Non-zero memory: Mem[0]=1.0, Mem[4]=2.0, Mem[8]=3.0, Mem[12]=4.0, Mem[16]=5.0, Mem[20]=6.0, Mem[24]=7.0, Mem[28]=8.0, Mem[32]=36.0, Mem[36]=204.0.

Issue width 1

```
                          ISSUE   EX       MEM      WB      COMMIT
Ld F1, 0(R1)              1       2        3-4      5       6
Add.d F0, F0, F1          2       6-7               8       9
Mult.d F2, F1, F1         3       6-9               10      11
Add.d F3, F3, F2          4       11-12             13      14
Addi R1, R1, 4            5       6                 7       15
Bne R1, R2, -6            6       8                         16
Ld F1, 0(R1)              10      11       12-13    14      17
Add.d F0, F0, F1          11      15-16             17      18
Mult.d F2, F1, F1         12      15-18             19      20
Add.d F3, F3, F2          13      20-21             22      23
Addi R1, R1, 4            14      15                16      24
Bne R1, R2, -6            15      17                        25
Ld F1, 0(R1)              16      17       18-19    20      26
Add.d F0, F0, F1          17      21-22             23      27
Mult.d F2, F1, F1         18      21-24             25      28
Add.d F3, F3, F2          19      26-27             28      29
Addi R1, R1, 4            20      21                24      30
Bne R1, R2, -6            21      25                        31
Ld F1, 0(R1)              22      25       26-27    29      32
Add.d F0, F0, F1          23      30-31             32      33
Mult.d F2, F1, F1         24      30-33             34      35
Add.d F3, F3, F2          25      35-36             37      38
Addi R1, R1, 4            26      27                30      39
Bne R1, R2, -6            27      31                        40
Ld F1, 0(R1)              28      31       32-33    35      41
Add.d F0, F0, F1          29      36-37             38      42
Mult.d F2, F1, F1         30      36-39             40      43
Add.d F3, F3, F2          31      41-42             43      44
Addi R1, R1, 4            32      33                36      45
Bne R1, R2, -6            33      37                        46
Ld F1, 0(R1)              34      37       38-39    41      47
Add.d F0, F0, F1          35      42-43             44      48
Mult.d F2, F1, F1         36      42-45             46      49
Add.d F3, F3, F2          37      47-48             49      50
Addi R1, R1, 4            38      39                42      51
Bne R1, R2, -6            39      43                        52
Ld F1, 0(R1)              40      43       44-45    47      53
Add.d F0, F0, F1          41      48-49             50      54
Mult.d F2, F1, F1         42      48-51             52      55
Add.d F3, F3, F2          43      53-54             55      56
Addi R1, R1, 4            44      45                48      57
Bne R1, R2, -6            45      49                        58
Ld F1, 0(R1)              46      49       50-51    53      59
Add.d F0, F0, F1          47      54-55             56      60
Mult.d F2, F1, F1         48      54-57             58      61
Add.d F3, F3, F2          49      59-60             61      62
Addi R1, R1, 4            50      51                54      63
Bne R1, R2, -6            51      55                        64
Sd F0, 0(R2)              57      58       65-66            65
Sd F3, 4(R2)              58      59       67-68            67

Squashed (wrong-path) instructions:
Sd F0, 0(R2)              7       8
Sd F3, 4(R2)              8
Ld F1, 0(R1)              52      55
Add.d F0, F0, F1          53
Mult.d F2, F1, F1         54
Add.d F3, F3, F2          55

Total cycles: 68   IPC: 0.74   mispredictions: 2
```

Issue width 4

```
                          ISSUE   EX       MEM      WB      COMMIT
Ld F1, 0(R1)              1       2        3-4      5       6
Add.d F0, F0, F1          1       6-7               8       9
Mult.d F2, F1, F1         1       6-9               10      11
Add.d F3, F3, F2          1       11-12             13      14
Addi R1, R1, 4            2       3                 4       14
Bne R1, R2, -6            2       5                         14
Ld F1, 0(R1)              7       8        9-10     11      14
Add.d F0, F0, F1          7       12-13             14      15
Mult.d F2, F1, F1         7       12-15             16      17
Add.d F3, F3, F2          7       17-18             19      20
Addi R1, R1, 4            8       9                 12      20
Bne R1, R2, -6            8       13                        20
Ld F1, 0(R1)              9       13       14-15    17      20
Add.d F0, F0, F1          9       18-19             20      21
Mult.d F2, F1, F1         9       18-21             22      23
Add.d F3, F3, F2          11      23-24             25      26
Addi R1, R1, 4            11      14                15      26
Bne R1, R2, -6            11      16                        26
Ld F1, 0(R1)              12      16       17-18    21      26
Add.d F0, F0, F1          12      22-23             24      27
Mult.d F2, F1, F1         12      22-25             26      27
Add.d F3, F3, F2          17      27-28             29      30
Addi R1, R1, 4            17      18                23      30
Bne R1, R2, -6            17      24                        30
Ld F1, 0(R1)              18      24       25-26    27      30
Add.d F0, F0, F1          18      28-29             30      31
Mult.d F2, F1, F1         18      28-31             32      33
Add.d F3, F3, F2          22      33-34             35      36
Addi R1, R1, 4            22      25                28      36
Bne R1, R2, -6            22      29                        36
Ld F1, 0(R1)              23      29       30-31    33      36
Add.d F0, F0, F1          23      34-35             36      37
Mult.d F2, F1, F1         23      34-37             38      39
Add.d F3, F3, F2          27      39-40             41      42
Addi R1, R1, 4            27      30                31      42
Bne R1, R2, -6            27      32                        42
Ld F1, 0(R1)              28      32       33-34    37      42
Add.d F0, F0, F1          28      38-39             40      43
Mult.d F2, F1, F1         28      38-41             42      43
Add.d F3, F3, F2          33      43-44             45      46
Addi R1, R1, 4            33      34                39      46
Bne R1, R2, -6            33      40                        46
Ld F1, 0(R1)              34      40       41-42    43      46
Add.d F0, F0, F1          34      44-45             46      47
Mult.d F2, F1, F1         34      44-47             48      49
Add.d F3, F3, F2          38      49-50             51      52
Addi R1, R1, 4            38      41                44      52
Bne R1, R2, -6            38      45                        52
Sd F0, 0(R2)              47      48       52-53            52
Sd F3, 4(R2)              47      49       54-55            54

Squashed (wrong-path) instructions:
Sd F0, 0(R2)              2       3
Sd F3, 4(R2)              2       4
Ld F1, 0(R1)              39      45
Add.d F0, F0, F1          39
Mult.d F2, F1, F1         39
Add.d F3, F3, F2          43
Addi R1, R1, 4            43
Bne R1, R2, -6            43
Ld F1, 0(R1)              44
Add.d F0, F0, F1          44
Mult.d F2, F1, F1         44

Total cycles: 55   IPC: 0.91   mispredictions: 2
```

Test 5: Beq/Bne, squashing, BTB aliasing (`tests/test5_branches.txt`)

A Beq-controlled loop, a taken forward Bne that skips a store, and two branches at word addresses 2 and 10, which share BTB entry 2, so the second branch is predicted with the wrong target. Wrong-path stores must never reach memory.

```
                # of rs     Cycles in EX    Cycles in Mem   # of FUs
Integer adder     4             1                             1
FP adder          2             2                             1
FP multiplier     2             4                             1
Load/store unit   4             1               3             1

ROB entries = 32
Issue width = 1   (overridden: 1, 2, 3, 4)
F1=1.5

Addi R1, R0, 3
Addi R2, R0, 0
Beq R1, R0, 4
Addi R2, R2, 10
Addi R1, R1, -1
Beq R0, R0, -4
Sd F1, 0(R0)
Bne R2, R3, 1
Sd F1, 4(R0)
Add.d F2, F1, F1
Beq R1, R0, 1
Addi R5, R0, 99
Sd F2, 8(R0)
```

Final state (all widths): R2=30, F1=1.5, F2=3.0.
Non-zero memory: Mem[8]=3.0.

Issue width 1

```
                          ISSUE   EX       MEM      WB      COMMIT
Addi R1, R0, 3            1       2                 3       4
Addi R2, R0, 0            2       3                 4       5
Beq R1, R0, 4             3       4                         6
Addi R2, R2, 10           4       5                 6       7
Addi R1, R1, -1           5       6                 7       8
Beq R0, R0, -4            6       7                         9
Beq R1, R0, 4             9       10                        11
Addi R2, R2, 10           10      11                12      13
Addi R1, R1, -1           11      12                13      14
Beq R0, R0, -4            12      13                        15
Beq R1, R0, 4             13      14                        16
Addi R2, R2, 10           14      15                16      17
Addi R1, R1, -1           15      16                17      18
Beq R0, R0, -4            16      17                        19
Beq R1, R0, 4             17      18                        20
Bne R2, R3, 1             20      21                        22
Add.d F2, F1, F1          23      24-25             26      27
Beq R1, R0, 1             24      25                        28
Sd F2, 8(R0)              27      28       29-31            29

Squashed (wrong-path) instructions:
Sd F1, 0(R0)              7
Addi R2, R2, 10           18
Sd F1, 4(R0)              21
Bne R2, R3, 1             25

Total cycles: 31   IPC: 0.61   mispredictions: 4
```

Issue width 2

```
                            ISSUE   EX       MEM      WB      COMMIT
Addi R1, R0, 3              1       2                 3       4
Addi R2, R0, 0              1       3                 4       5
Beq R1, R0, 4               2       4                         5
Addi R2, R2, 10             2       5                 6       7
Addi R1, R1, -1             3       6                 7       8
Beq R0, R0, -4              3       7                         8
Beq R1, R0, 4               9       10                        11
Addi R2, R2, 10             9       11                12      13
Addi R1, R1, -1             10      12                13      14
Beq R0, R0, -4              10      13                        14
Beq R1, R0, 4               11      14                        15
Addi R2, R2, 10             11      15                16      17
Addi R1, R1, -1             12      16                17      18
Beq R0, R0, -4              13      17                        18
Beq R1, R0, 4               14      18                        19
Bne R2, R3, 1               20      21                        22
Add.d F2, F1, F1            23      24-25             26      27
Beq R1, R0, 1               23      24                        27
Sd F2, 8(R0)                26      27       28-30            28

Squashed (wrong-path) instructions:
Sd F1, 0(R0)                4       5
Bne R2, R3, 1               4
Sd F1, 4(R0)                5       6
Add.d F2, F1, F1            5       6-7
Beq R1, R0, 1               6
Addi R5, R0, 99             6
Sd F2, 8(R0)                7
Addi R2, R2, 10             15
Addi R1, R1, -1             16
Beq R0, R0, -4              17
Beq R1, R0, 4               18
Sd F1, 4(R0)                20      21
Add.d F2, F1, F1            21
Beq R1, R0, 1               21
Bne R2, R3, 1               24

Total cycles: 30   IPC: 0.63   mispredictions: 4
```

Test 6: Structural hazards (`tests/test6_structural.txt`)

A 4-entry ROB, one reservation station per unit and a 2-entry load/store queue. Issue must stall in order when any of these is full.

```
                # of rs     Cycles in EX    Cycles in Mem   # of FUs
Integer adder     1             1                             1
FP adder          1             3                             1
FP multiplier     1             6                             1
Load/store unit   2             1               3             1

ROB entries = 4
Issue width = 1   (overridden: 1, 2, 3, 4)
R1=4, F1=2.0, F2=3.0
Mem[4]=5, Mem[8]=6

Ld F3, 0(R1)
Ld F4, 4(R1)
Mult.d F5, F3, F4
Add.d F6, F5, F1
Mult.d F7, F2, F2
Addi R2, R1, 8
Add.d F8, F7, F6
Sd F8, 0(R2)
Addi R3, R2, 4
Sd F5, 0(R3)
Sub.d F9, F8, F5
```

Final state (all widths): R1=4, R2=12, R3=16, F1=2.0, F2=3.0, F3=5.0, F4=6.0, F5=30.0, F6=32.0, F7=9.0, F8=41.0, F9=11.0.
Non-zero memory: Mem[4]=5.0, Mem[8]=6.0, Mem[12]=41.0, Mem[16]=30.0.

Issue width 1

```
                          ISSUE   EX       MEM      WB      COMMIT
Ld F3, 0(R1)              1       2        3-5      6       7
Ld F4, 4(R1)              2       3        6-8      9       10
Mult.d F5, F3, F4         3       10-15             16      17
Add.d F6, F5, F1          4       17-19             20      21
Mult.d F7, F2, F2         10      11-16             17      22
Addi R2, R1, 8            11      12                13      23
Add.d F8, F7, F6          17      21-23             24      25
Sd F8, 0(R2)              21      22       26-28            26
Addi R3, R2, 4            22      23                25      27
Sd F5, 0(R3)              23      26       29-31            29
Sub.d F9, F8, F5          25      26-28             29      30

Total cycles: 31   IPC: 0.35   mispredictions: 0
```

Issue width 4

```
                          ISSUE   EX       MEM      WB      COMMIT
Ld F3, 0(R1)              1       2        3-5      6       7
Ld F4, 4(R1)              1       3        6-8      9       10
Mult.d F5, F3, F4         1       10-15             16      17
Add.d F6, F5, F1          1       17-19             20      21
Mult.d F7, F2, F2         10      11-16             17      21
Addi R2, R1, 8            10      11                12      21
Add.d F8, F7, F6          17      21-23             24      25
Sd F8, 0(R2)              21      22       25-27            25
Addi R3, R2, 4            21      22                23      25
Sd F5, 0(R3)              21      24       28-30            28
Sub.d F9, F8, F5          25      26-28             29      30

Total cycles: 30   IPC: 0.37   mispredictions: 0
```

Test 7: Handout sample input (`tests/test7_handout_sample.txt`)

The configuration and initial values of the sample input in the handout. The handout's `Bne R2, R3, -3` would loop forever because R2 never changes, so an `Addi` was added to end the loop. Note that `8(R1)` = 18 is not word-aligned; the memory ignores the two low address bits.

```
                        # of rs       Cycles in EX      Cycles in Mem         # of FUs
   Integer adder           2             1                                     1
   FP adder                3             3                                     1
   FP multiplier           2             20                                    1
   Load/store unit         3             1                     4               1

ROB entries = 128
Issue width = 2
R1=10, R2=20, F2=30.1
Mem[4]=1, Mem[8]=2, Mem[12]=3.4

Add.d F1, F2, F3
Ld F4, 8(R1)
Addi R2, R2, -10
Bne R2, R3, -4
Mult.d F5, F1, F2
```

Final state (all widths): R1=10, F1=30.1, F2=30.1, F5=906.01.
Non-zero memory: Mem[4]=1.0, Mem[8]=2.0, Mem[12]=3.4.

Issue width 1

```
                           ISSUE   EX       MEM      WB      COMMIT
Add.d F1, F2, F3           1       2-4              5       6
Ld F4, 8(R1)               2       3        4-7     8       9
Addi R2, R2, -10           3       4                6       10
Bne R2, R3, -4             4       7                        11
Add.d F1, F2, F3           9       10-12            13      14
Ld F4, 8(R1)               10      11       12-15   16      17
Addi R2, R2, -10           11      12               14      18
Bne R2, R3, -4             12      15                       19
Mult.d F5, F1, F2          17      18-37            38      39

Total cycles: 39   IPC: 0.23   mispredictions: 2
```

Issue width 2

```
                           ISSUE   EX       MEM      WB      COMMIT
Add.d F1, F2, F3           1       2-4              5       6
Ld F4, 8(R1)               1       2        3-6     7       8
Addi R2, R2, -10           2       3                4       8
Bne R2, R3, -4             2       5                        9
Add.d F1, F2, F3           7       8-10             11      12
Ld F4, 8(R1)               7       8        9-12    13      14
Addi R2, R2, -10           8       9                10      14
Bne R2, R3, -4             8       11                       15
Mult.d F5, F1, F2          13      14-33            34      35

Total cycles: 35   IPC: 0.26   mispredictions: 2
```

Notes on the tables

The tables above show the features under test:

- **Test 1, W=4:** four instructions are issued in each of cycles 1, 2 and
    3. Two integer adds start EX together in cycle 2 (2 integer FUs), and write-backs are serialized on the single CDB in cycles 3 to 14.
- **Test 2:** `Mult.d F3, F2, F2` uses the first F2 (3.0) even though `Add.d F2, F1, F1` writes F2 later. `Sub.d F4, F3, F2` uses the new F2 (4.0), so F4 = 9 - 4 = 5. Register renaming removes the WAR and WAW hazards.
- **Test 3:** `Ld F6, 12(R1)` gets 21.5 forwarded from `Sd F5, 4(R2)` (same address, 12) with a 1-cycle MEM. `Ld F8, 8(R1)` waits for the multiplier's result to reach the store, then is forwarded 3.0. `Ld F4, 4(R1)` reads memory because no older store matches its address.
- **Test 4:** the first `Bne` is predicted not-taken (empty BTB) and mispredicts. The next fetch comes 2 cycles after it resolves. Later iterations are predicted taken and fetched speculatively. The loop exit mispredicts again, and the wrong-path loop instructions appear in the squashed list.
- **Test 5:** the wrong-path `Sd F1, 0(R0)` and `Sd F1, 4(R0)` are issued and even calculate their addresses, but they are squashed and never write memory (only `Mem[8]` is non-zero). `Beq R1, R0, 1` at word address 10 uses BTB entry 2, which belongs to the branch at address 2, and is predicted taken with the wrong target (7 instead of 12).
- **Test 6:** with a 4-entry ROB and one RS per unit, issue stalls in order long before W matters (31 vs. 30 cycles).